

Características básicas del lenguaje	2
Estructuras de control (condicionales, bucles)	2
Condicionales	2
Bucles (Loops)	4
Tipos de datos (primitivos y referencias)	4
Primitivos	5
Referencias (Objects)	5
Funciones (declaradas, expresadas, flechas)	5
Objetos y arrays	6
Eventos y manipulación del DOM	7
Asincronía (callbacks, promesas, async/await)	7

Características básicas del lenguaje

JavaScript es un lenguaje de programación de alto nivel con varias características clave:

- Interpretado: El código se ejecuta línea por línea en el navegador o en el servidor (con Node.js), sin necesidad de una compilación previa a lenguaje máquina.
- Dinámico: Esto significa que no es necesario declarar el tipo de una variable al crearla. Una misma variable puede almacenar un número, luego un texto, y luego un objeto. Además, los arrays en JavaScript son dinámicos, pudiendo crecer y reducir su tamaño después de ser creados y contener una mezcla de diferentes tipos de valor .
- Multiparadigma: Soporta diferentes estilos de programación, como la programación orientada a objetos, la programación funcional y la imperativa.
- Débilmente tipado: Realiza conversiones de tipo de manera implícita, lo que puede ser útil pero también requiere atención para evitar comportamientos inesperados.

Estructuras de control (condicionales, bucles)

Las estructuras de control determinan el flujo de ejecución de un programa, permitiendo tomar decisiones y repetir acciones.

Condicionales

Permiten ejecutar diferentes bloques de código según si se cumple una condición específica. La estructura más común es if...else :

```
let nombreUsuario = "María";
if (nombreUsuario === "María") {
  console.log(`¡Hola, ${nombreUsuario}! Bienvenida de nuevo.`);
} else {
  console.log(`Hola, visitante.`);
}
```

Bucles (Loops)

Repiten un bloque de código mientras se cumpla una condición. Los principales tipos son :

- for: Se utiliza cuando se sabe el número exacto de iteraciones. Se compone de una inicialización, una condición y un incremento .

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

- while: Ejecuta el código mientras la condición sea verdadera. Es útil cuando no se conoce el número de iteraciones de antemano.

```
let i = 0;  
while (i <= 5) {  
  console.log(i);  
  i++;  
}
```

- do...while: Similar a while, pero garantiza que el bloque de código se ejecute al menos una vez, ya que la condición se evalúa al final.

```
let i = 0;  
do {  
  console.log(i);  
  i++;  
} while (i <= 5);
```

Tipos de datos (primitivos y referencias)

JavaScript maneja dos categorías principales de tipos de datos, que se comportan de manera diferente en cuanto a cómo se almacenan y se copian .

Primitivos

Son datos inmutables y se copian "por valor". Al asignar una variable primitiva a otra, se crea una copia independiente. Cambiar una no afecta a la otra . Los tipos primitivos son :

- Undefined
- Boolean (true / false)
- Number (para números enteros y de punto flotante)
- String (para texto, también inmutable)
- BigInt (para números enteros muy grandes)
- Symbol (para crear identificadores únicos)
- null

Referencias (Objects)

Son objetos (como Object, Array, Function) y se copian "por referencia". Al asignar un objeto a otra variable, ambas apuntan a la misma ubicación en la memoria (el heap) . Si se modifica el objeto a través de una variable, el cambio se refleja en todas las demás que referencian al mismo objeto.

```
const referencia = [1];
const refCopy = referencia;
refCopy.push(2);
console.log(referencia); // Resultado: [1, 2], ya que ambas variables apuntan al mismo array
```

Funciones (declaradas, expresadas, flechas)

Las funciones son bloques de código reutilizables. Existen varias formas de crearlas :

- Declaración de función: Se crea con la palabra clave function y un nombre. Tienen la característica de "elevarse" (hoisting), lo que permite llamarlas antes de su declaración en el código.

```
function saludar(nombre) {
  return `Hola, ${nombre}!`;
}
```

- **Expresión de función:** Se asigna una función anónima (sin nombre) a una variable. No se elevan, por lo que solo se pueden usar después de su definición .

```
const saludar = function(nombre) {  
  return `Hola, ${nombre}!`;  
};
```

Función flecha (Arrow Function): Es una sintaxis más concisa introducida en ES6. No tienen su propio `this` (heredan el del contexto padre), lo que las hace ideales para callbacks y métodos de arrays. Su sintaxis puede acortarse si tienen un solo parámetro o una sola expresión de retorno .

```
// Forma estándar  
const saludar = (nombre) => {  
  return `Hola, ${nombre}!`;  
};  
// Forma abreviada  
const saludar = nombre => `Hola, ${nombre}!`;
```

Objetos y arrays

- **Objetos:** Son colecciones de propiedades, que son pares de clave (nombre) y valor . Un valor puede ser cualquier tipo de dato, incluyendo otros objetos o funciones (en ese caso, la propiedad se llama método). Se crean comúnmente con llaves `{}`.

```
const persona = {  
  nombre: ["Bob", "Smith"],  
  edad: 32,  
  bio() {  
    console.log(`${this.nombre[0]} ${this.nombre[1]} tiene ${this.edad} años.`);  
  }  
};  
console.log(persona.nombre[0]); // Acceso a propiedad con "."  
persona.bio(); // Llamada a método
```

- **Arrays:** Son objetos especializados para almacenar listas ordenadas de valores, que pueden ser de cualquier tipo . Se crean con corchetes `[]`. Son dinámicos y tienen métodos útiles como `push()`, `pop()`, `sort()`, etc.

```
const numeros = [43, "hola", -2.5, true];  
numeros.push(100); // Añade un elemento al final  
console.log(numeros[0]); // Acceso por índice (0)
```

Eventos y manipulación del DOM

El DOM (Document Object Model) es una representación en forma de árbol de la estructura de una página web. JavaScript permite interactuar con él para crear páginas dinámicas .

- **Manipulación del DOM:** Se refiere a la capacidad de JavaScript para seleccionar elementos HTML (por ejemplo, con `document.getElementById`, `document.querySelector`), modificar su contenido (`element.textContent`), cambiar sus atributos o estilos CSS (`element.style.color = "red";`), y crear o eliminar elementos.
- **Eventos:** Son acciones que ocurren en la página (como un clic, pasar el mouse sobre un elemento, o cargar la página). JavaScript puede "escuchar" estos eventos y ejecutar una función (llamada event handler o listener) en respuesta . Por ejemplo:

```
document.getElementById("miBoton").addEventListener("click", function() {  
  alert("¡El botón fue presionado!");  
});
```

Asincronía (callbacks, promesas, async/await)

JavaScript es de un solo hilo (single-threaded), lo que significa que solo puede hacer una cosa a la vez. Para manejar operaciones que tardan tiempo (como solicitudes a un servidor, leer un archivo, o un temporizador) sin bloquear la ejecución del resto del código, se utilizan técnicas asíncronas.

- **Callbacks:** Es una función que se pasa como argumento a otra función y se ejecuta cuando una tarea asíncrona se completa. Es la forma más básica, pero puede llevar a un "infierno de callbacks" si se anidan demasiado.

```
setTimeout(() => {  
  console.log("Esto se ejecuta después de 1 segundo.");  
}, 1000);
```

- **Promesas:** Son objetos que representan el resultado eventual de una operación asíncrona (éxito o fracaso). Tienen estados (pendiente, cumplida, rechazada) y permiten encadenar operaciones de forma más legible con `.then()` y `.catch()`.
- **async / await:** Es una sintaxis moderna que hace que el código asíncrono se vea y se comporte más como código síncrono, facilitando la lectura y el manejo de errores. Una función declarada con `async` devuelve implícitamente una promesa, y la palabra clave `await` pausa la ejecución de la función hasta que la promesa se resuelva.

JS

JAVASCRIPT

JavaScript es un lenguaje de programación o de secuencias de comandos que te permite implementar funciones complejas en páginas web.

1 Características básicas del lenguaje

No requiere ser compilado a código de máquina antes de ejecutarse.
Tipo dinámico: Las variables no se declaran con un tipo de dato fijo (como en C++ o Java).

Paradigma

No te limita a una sola forma de programar.
El motor de JavaScript utiliza un solo hilo (single-thread) con un bucle de eventos (Event Loop) para la concurrencia.

2 Estructuras de control

Las estructuras de control en JavaScript dirigen el flujo de ejecución del código. Las condicionales (if, else, switch) toman decisiones y ejecutan bloques de código según condiciones lógicas.

Bucles (Estructuras de Repetición)
Permiten ejecutar un bloque de código varias veces mientras se cumpla una condición específica.

3 Tipos de datos

Los primitivos son inmutables y se almacenan directamente en la memoria stack. Los de referencia son colecciones dinámicas guardadas en el heap, mientras que su stack solo almacena un apuntador (dirección de memoria).

Tipos de datos Primitivos
Son inmutables (su valor no se puede modificar directamente, solo reasignar la variable).
Tipos de datos de Referencia
Son mutables y corresponden a los Objetos (incluyendo Arrays, Funciones, Fechas, etc.).

4 Funciones

declaradas (llevan nombre y sufren hoisting, permitiendo llamarlas antes de definir las), expresadas (asignadas a variables, sin hoisting) y flecha (versión moderna y concisa que no enlaza su propio this).

Funciones Declaradas (Function Declarations)
Utilizan la palabra clave function seguida obligatoriamente de un nombre.
Funciones Expresadas (Function Expressions)
Se crean asignando una función anónima (o con nombre) a una variable.
Funciones Flecha (Arrow Functions)
Introducidas en ES6, ofrecen una sintaxis más corta y limpia.

JS

JAVASCRIPT

JavaScript es un lenguaje de programación o de secuencias de comandos que te permite implementar funciones complejas en páginas web.

5 Objetos y arrays

lecciones desordenadas de pares clave-valor ideales para modelar entidades complejas, mientras que los arrays son listas ordenadas que almacenan elementos secuencialmente mediante índices numéricos.

Un array almacena múltiples valores en una sola variable.

Arrays de Objetos
La combinación más común en JavaScript es el array de objetos.

6 Eventos y manipulación del DOM

la manipulación del DOM y el manejo de eventos. Estas habilidades son cruciales para crear interfaces de usuario dinámicas y diseñar páginas web interactivas.

Creación de elementos responsivos
Modificación dinámica de páginas web
Desarrollo de aplicaciones web avanzadas

7 Asincronía

permite que el código ejecute tareas costosas (como consultas a bases de datos o peticiones HTTP) sin bloquear el hilo principal.

Se gestiona mediante callbacks (funciones de retorno), promesas (objetos que representan el resultado de una tarea) o async/await (sintaxis moderna que simplifica la lectura del código).